

Strings and Unicode — A Readable Companion to the Elixir Docs

Chapter 7 · checked against Elixir 1.20 · examples run on Elixir 1.14 · Sep 26, 2026

1. Why “string” means three different things

An Elixir string is a UTF-8 encoded binary. That one sentence hides three separate ways of counting:

- **Bytes:** what the binary actually holds, what a database column limit or a network protocol counts.
- **Code points:** Unicode characters: é can be one code point, or two (e plus a combining accent).
- **Graphemes:** what a person sees as one character: 🍌 is one grapheme made of two code points and eight bytes.

String functions work in graphemes; `byte_size/1`, `binary_part/3` and pattern matching with `<<...>>` work in bytes. Most string bugs come from counting in one unit and limiting in another. English test data hides them, because for ASCII all three counts are the same.

Elixir also has *charlists*, lists of code points (`~c"abc"`, or `'abc'` in older code), used mainly to talk to Erlang libraries.

2. The building blocks

Function	Counts or works in	Cost
<code>byte_size/1</code>	Bytes	Constant
<code>String.length/1</code>	Graphemes	Linear: walks the whole string
<code>String.slice/3</code> , <code>String.at/2</code>	Graphemes	Linear
<code>binary_part/3</code> , <code><<head::binary-size(n), _::binary>></code>	Bytes	Constant; can cut a character in half
<code>String.graphemes/1</code> / <code>String.codepoints/1</code>	Graphemes / code points	Linear
<code>String.valid?/1</code>	Checks the bytes are valid UTF-8	Linear
<code>String.normalize/2</code> , <code>String.equivalent?/2</code>	Unicode normal forms (NFC, NFD...)	Linear
<code>String.upcase/2</code> , <code>downcase/2</code>	Graphemes, with <code>:default</code> , <code>:ascii</code> , <code>:turkic</code> , <code>:greek</code> modes	Linear

Strings versus charlists

	String	Charlist
Literal	<code>"abc"</code>	<code>~c"abc"</code> (older code: <code>'abc'</code>)
Representation	UTF-8 binary	List of integer code points
Memory	About 1 byte per ASCII character	2 words (16 bytes) per character (Erlang book, chapter 3)
Where you meet it	Elixir code	Erlang APIs (<code>:inet</code> , <code>:io_lib</code> , <code>:os.cmd...</code>)
Convert	<code>List.to_string/1</code>	<code>String.to_charlist/1</code>

3. How it actually works

Equal-looking is not equal

Unicode allows the same visible text to be encoded in different ways. "café" typed on one keyboard can be `c a f é` (one code point for `é`), while text from another system can be `c a f e ΅` (e plus a combining accent). They look identical, but `==` compares bytes and returns `false`. `String.equivalent?/2` compares normalised forms; `String.normalize(s, :nfc)` at the boundary makes all later comparisons reliable.

Regular expressions default to bytes

Elixir regexes are PCRE. Without the `u` modifier, `\w`, `.` and character classes work on bytes and ASCII rules, so `~r/^\w+$/` does not match `"café"`. With `~r/^\w+$/u` it does.

An integer list may print as text

`I0.inspect/1` prints a list of integers as a charlist when all of them are printable. `[7, 8, 9]` prints as `~c"\a\b\t"` (`'\a\b\t'` on older versions). Nothing is wrong with the data; only the display changed. Pass `charlists: :as_lists` to `inspect` to see numbers.

4. Plausible but wrong

4.1 Limiting bytes with String.length

```
def fits_column?(name), do: String.length(name) <= 255
def truncate(name), do: String.slice(name, 0, 255)
```

The database limit is 255 *bytes*; this checks graphemes. A name of 150 `é` characters passes the check at 150 graphemes but is 300 bytes (verified), and the insert fails, or the column silently truncates it. Fix: check `byte_size/1`. To truncate safely, cut on a grapheme boundary: take graphemes while the running byte total stays under the limit.

4.2 Cutting bytes in the middle of a character

```
preview = binary_part(body, 0, min(byte_size(body), 151))
```

Taking a fixed number of bytes can split a multi-byte character, leaving invalid UTF-8 (verified: `String.valid?/1` returns `false`). JSON encoders then raise, and some clients display garbage. Fix: truncate by graphemes (`String.slice/3`) when the limit is visual, or back off to a valid boundary when the limit is in bytes.

4.3 Comparing text from different sources

```
def same_city?(a, b), do: String.downcase(a) == String.downcase(b)
```

"café" from a web form and "café" from an import can differ in encoding, so this returns `false` for identical-looking values (verified: `== false`, `String.equivalent?/2 true`). Duplicates slip past uniqueness checks. Fix: normalise all incoming text once, with `String.normalize(s, :nfc)`, before storing or comparing.

4.4 ASCII-only regex on Unicode input

```
def valid_username?(u), do: Regex.match?(~r/^\w{3,20}$/u, u)
```

"josé" and every non-Latin name are rejected, because without `u` the regex works on bytes with ASCII rules (verified). Fix: `~r/^\w{3,20}$/u`. Decide deliberately which characters you allow: `\w` with `u` accepts letters from every script.

4.5 Joining an Erlang charlist with <>

```
{:ok, {a, b, c, d}} = get_peer_ip(socket)
"client " <> :inet.ntoa({a, b, c, d})
```

`:inet.ntoa/1` returns a charlist, `~c"10.0.0.1"`, and `<>` requires binaries, so this raises `ArgumentError` at runtime (verified). String interpolation hides the problem because it converts charlists automatically, so the same value works in one place and fails in another. Fix: convert at the boundary: `:inet.ntoa(ip) |> List.to_string()`.

4.6 String.length as an emptiness test

```
def blank?(s), do: String.length(String.trim(s)) == 0
```

`String.length/1` walks the entire string to count graphemes. On large inputs checked often, that is wasted work. Fix: `String.trim(s) == ""`, or `byte_size(s) == 0` for an exact empty check.

4.7 Upper-casing for keys and comparisons

```
key = String.upcase(code)    # "istanbul" -> "ISTANBUL"
```

Default Unicode rules are right for most languages but not all: Turkish distinguishes dotted and dotless i, and `String.upcase/2` has a `:turkic` mode for that. Case-folding user text to build lookup keys can merge or split words unexpectedly. Fix: for identifiers you control, restrict input to ASCII and use `String.upcase(code, :ascii)`. For human text, compare normalised strings and choose the mode deliberately.

5. Review checklist and sources

- ☐ Byte limits (database columns, protocols) are checked with `byte_size/1`, not `String.length/1`
- ☐ Truncation never splits a character; results are valid UTF-8
- ☐ Incoming text is normalised (NFC) before storage or comparison
- ☐ Regexes that see user text use the `u` modifier
- ☐ Charlists from Erlang APIs are converted with `List.to_string/1` at the boundary
- ☐ Emptiness is tested with `== ""`, not `String.length/1`
- ☐ Case conversion for keys uses an explicit mode (`:ascii` for identifiers)

Sources

- String: <https://hexdocs.pm/elixir/String.html>
- Binaries, strings, and charlists: <https://hexdocs.pm/elixir/binaries-strings-and-charlists.html>
- Regex: <https://hexdocs.pm/elixir/Regex.html>
- Erlang book, chapter 3 (Binaries and Memory): <https://github.com/ayarodionov/Learning-Erlang-from-Claude>